

```

async def search_ngo_by_name(name: str, limit: int = 10, db_path: str = DB_PATH) -> dict:
    conn = _get_conn(db_path)
    try:
        cursor = conn.execute(
            "SELECT rowid, * FROM ngos WHERE rowid IN "
            "(SELECT rowid FROM ngos_fts WHERE ngos_fts MATCH ?) "
            "LIMIT ?",
            (f'"{name}"', limit),
        )
        rows = cursor.fetchall()
        results = [_row_to_dict(r) for r in rows]

        if not results:
            cursor = conn.execute(
                "SELECT rowid, * FROM ngos WHERE name LIKE ? LIMIT ?",
                (f"%{name}%", limit),
            )
            rows = cursor.fetchall()
            results = [_row_to_dict(r) for r in rows]

        return {"results": results, "total": len(results), "query": name}
    @router.post("/tools/{tool_name}")
    async def call_tool(tool_name: str, request: Request):
        if tool_name not in TOOL_REGISTRY:
            return JSONResponse(
                status_code=404,
                content={
                    "error": f"Tool '{tool_name}' not found",
                    "available_tools": list(TOOL_REGISTRY.keys()),
                },
            )

        body = await request.json()
        fn = TOOL_REGISTRY[tool_name]

        sig = inspect.signature(fn)
        valid_params = set(sig.parameters.keys())
        kwargs = {k: v for k, v in body.items() if k in valid_params}

        try:
            result = await fn(**kwargs)
            return {"tool": tool_name, "result": result}
        except TypeError as e:
            return JSONResponse(
                status_code=400,
                content={"error": f"Invalid arguments: {str(e)}", "tool": tool_name},
            )
        except Exception as e:
            return JSONResponse(
                status_code=500,
                content={"error": f"Tool execution failed: {str(e)}", "tool": tool_name},
            )

def store(df: pd.DataFrame, db_path: str = DB_PATH) -> None:
    os.makedirs(os.path.dirname(db_path), exist_ok=True)
    conn = sqlite3.connect(db_path)
    df.to_sql("ngos", conn, if_exists="replace", index=False)
    conn.execute("DROP TABLE IF EXISTS ngos_fts")
    conn.execute(
        "CREATE VIRTUAL TABLE ngos_fts USING fts5("
        "name, purpose, aims_objectives_mission, "
        "content='ngos', content_rowid='rowid')"
    )
    conn.execute(
        "INSERT INTO ngos_fts(rowid, name, purpose, aims_objectives_mission) "
        "SELECT rowid, name, purpose, aims_objectives_mission FROM ngos"
    )
    conn.commit()
    conn.close()

def run_etl(csv_path: str = CSV_PATH, db_path: str = DB_PATH) -> int:
    df = load(csv_path)
    df, dupes = clean(df)
    store(df, db_path)
    print(f"ETL complete: {len(df)} records -> {db_path}")
    if dupes:
        print(f" {dupes} duplicate names removed")
    return len(df)

async def check_fcra_status(
    registration_number: Optional[str] = None,
    ngo_name: Optional[str] = None,
    state: Optional[str] = None,

```

```
) -> dict:
    """Check FCRA registration status from the FCRA portal.

    Tries to scrape fcraonline.nic.in for registration details.
    Returns a dict with status info or an error message.
    """
    result = {
        "source": "fcraonline.nic.in",
        "registration_number": registration_number,
        "ngo_name": ngo_name,
        "registered": None,
        "status": None,
        "expiry_date": None,
        "fetch_method": "scrape",
        "error": None,
    }

    try:
        async with httpx.AsyncClient(timeout=30.0, follow_redirects=True) as client:
            headers = {"User-Agent": USER_AGENTS[0]}
```